Evolutionary Computation lab

Problem description

Problem that we are using to benchmark different algorithms is modified version of **TSP**.

In this problem each nodes are placed in euclidean space, with additional cost for visiting each one of them.

Our goal is to visit half (rounded up) nodes, and go back to the start is such way that the travel cost is minimized (calculating euclidean distance and with added cost)

Data

	X coordinate	Y coordinate	cost
0	1355	1796	496
1	2524	387	414
2	2769	430	500
3	3131	1199	1133
4	661	87	903
5	628	974	1081
6	1748	185	1859
7	3650	1882	945
8	3474	1230	1389
9	2662	1405	776

Evolutionary Computation lab

Dawid Siera id:156044

Anatol Kaczmarek id:156038

Source code is available on https://github.com/tole-k/evolutionary-computation.git

Problem description

Problem that we are using to benchmark different algorithms is modified version of **TSP**.

In this problem each nodes are placed in euclidean space, with additional cost for visiting each one of them.

Our goal is to visit half (rounded up) nodes, and go back to the start is such way that the travel cost is minimized (calculating euclidean distance and with added cost)

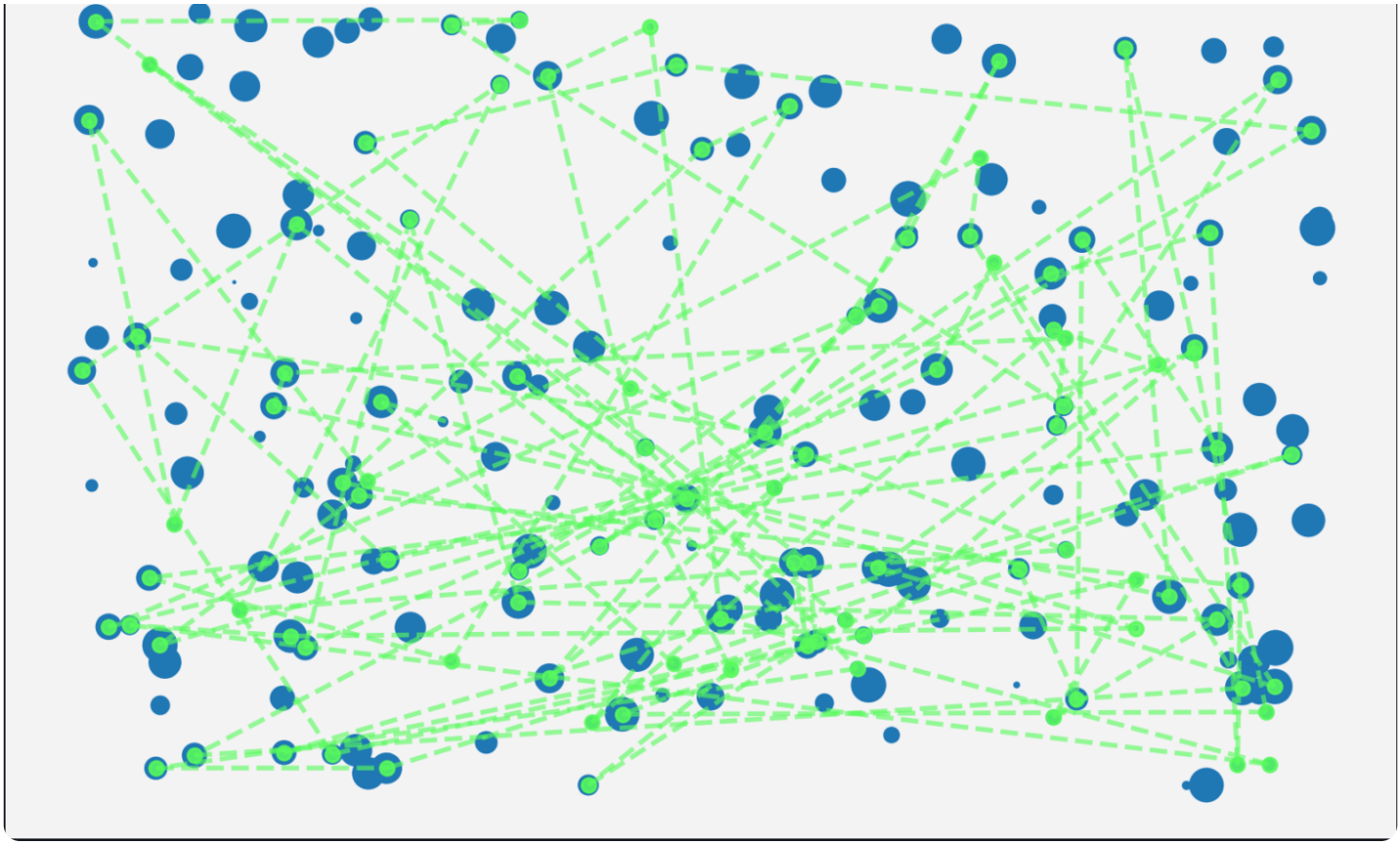
Greedy algorithms

Random

Pseudocode

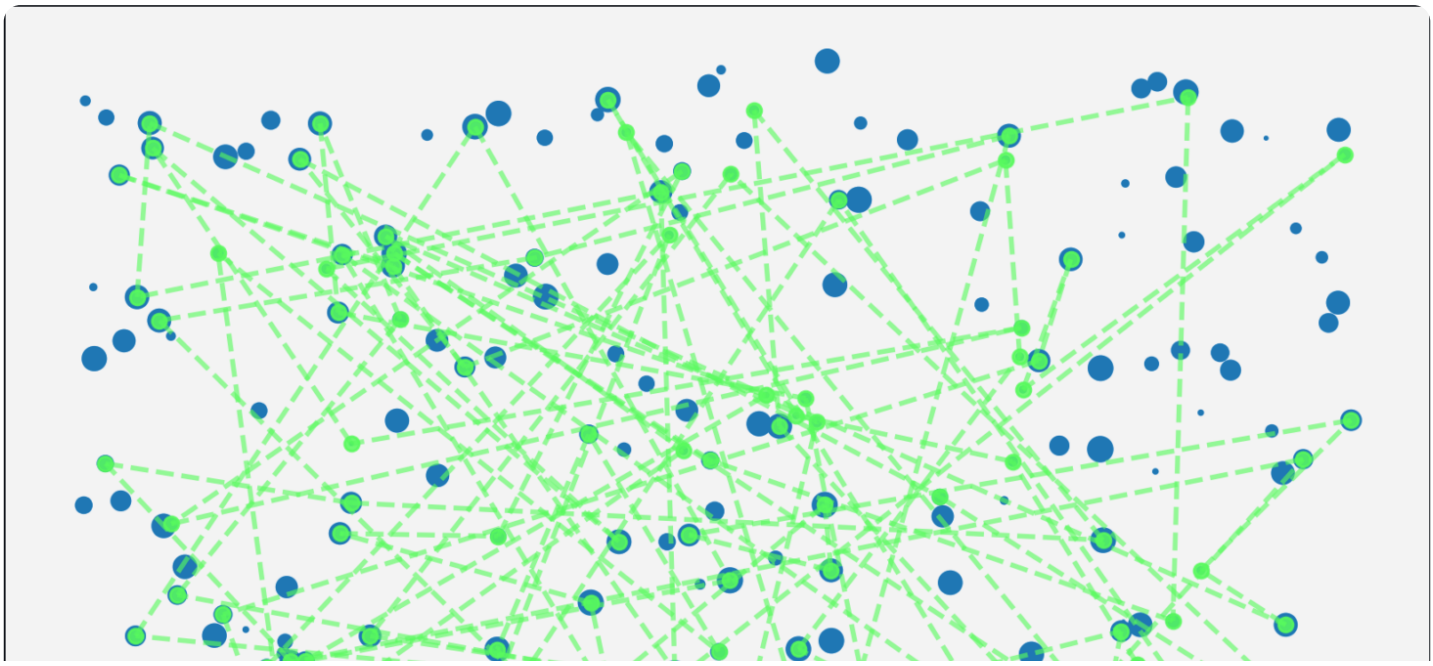
```
random (points)
    all_points = points
    all_points = shuffle (all_points)
    return all_points[:len(points)+1//2]
```

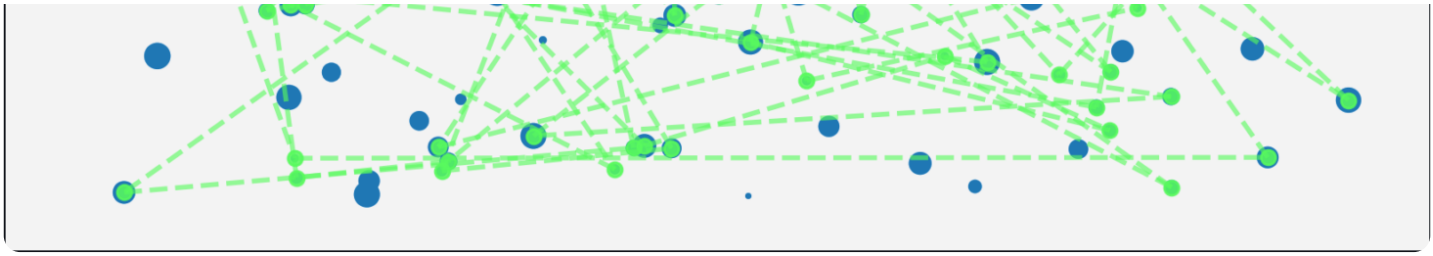
TSP A



Best found path: [92, 176, 18, 80, 101, 10, 29, 129, 25, 138, 169, 48, 123, 114, 23, 105, 189, 100, 6, 113, 40, 147, 47, 157, 171, 39, 32, 162, 37, 30, 124, 135, 178, 185, 74, 50, 49, 128, 35, 46, 161, 38, 44, 145, 4, 81, 180, 197, 78, 11, 54, 175, 136, 183, 153, 79, 53, 70, 97, 109, 122, 190, 13, 102, 144, 65, 20, 160, 191, 26, 19, 134, 143, 117, 52, 58, 133, 71, 89, 68, 94, 194, 148, 132, 9, 130, 195, 149, 55, 106, 96, 5, 168, 21, 119, 1, 112, 103, 0, 184]

TSP B





Best found path: [13, 1, 55, 121, 72, 157, 90, 28, 194, 160, 67, 103, 169, 77, 113, 124, 0, 65, 186, 42, 192, 68, 73, 29, 100, 120, 87, 126, 96, 17, 48, 166, 57, 56, 41, 37, 78, 80, 122, 82, 182, 123, 36, 164, 38, 112, 64, 3, 12, 163, 172, 193, 168, 175, 91, 191, 137, 50, 18, 152, 118, 178, 143, 93, 145, 173, 114, 7, 60, 62, 174, 83, 198, 190, 75, 189, 142, 180, 89, 156, 51, 115, 125, 109, 81, 53, 134, 32, 79, 153, 127, 58, 35, 71, 33, 108, 61, 171, 19, 25]

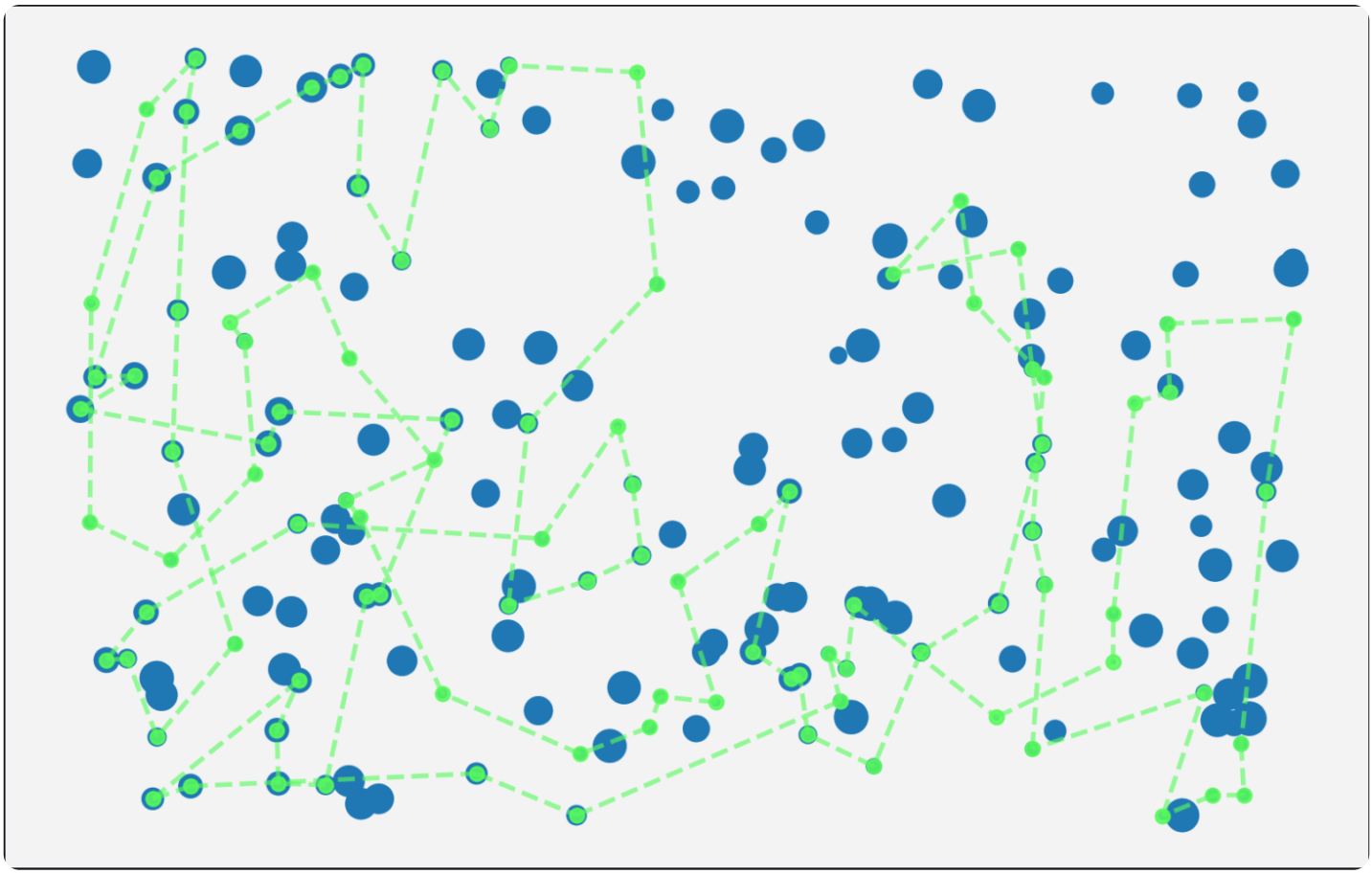
NN to last point

Pseudocode

```
find_closest(point, points, distance_matrix):
    closest_distance = infinity
    closest_point = point

    for each candidate_point in points:
        current_distance = distance_matrix[point, candidate_point] + candidate_po

        if current_distance < closest_distance:
            closest_distance = current_distance
            closest_point = candidate_point

    return closest_point, closest_distance



greedy_nn_to_last_point(points, starting_point, distance_matrix):
    last_point = starting_point
    tsp_path = [last_point]
    not_visited_points = points
    remove starting_point from not_visited_points

    for i from 1 to ceil(length(points) / 2):
        closest_point, _ = find_closest(last_point, not_visited_points, distance_
```

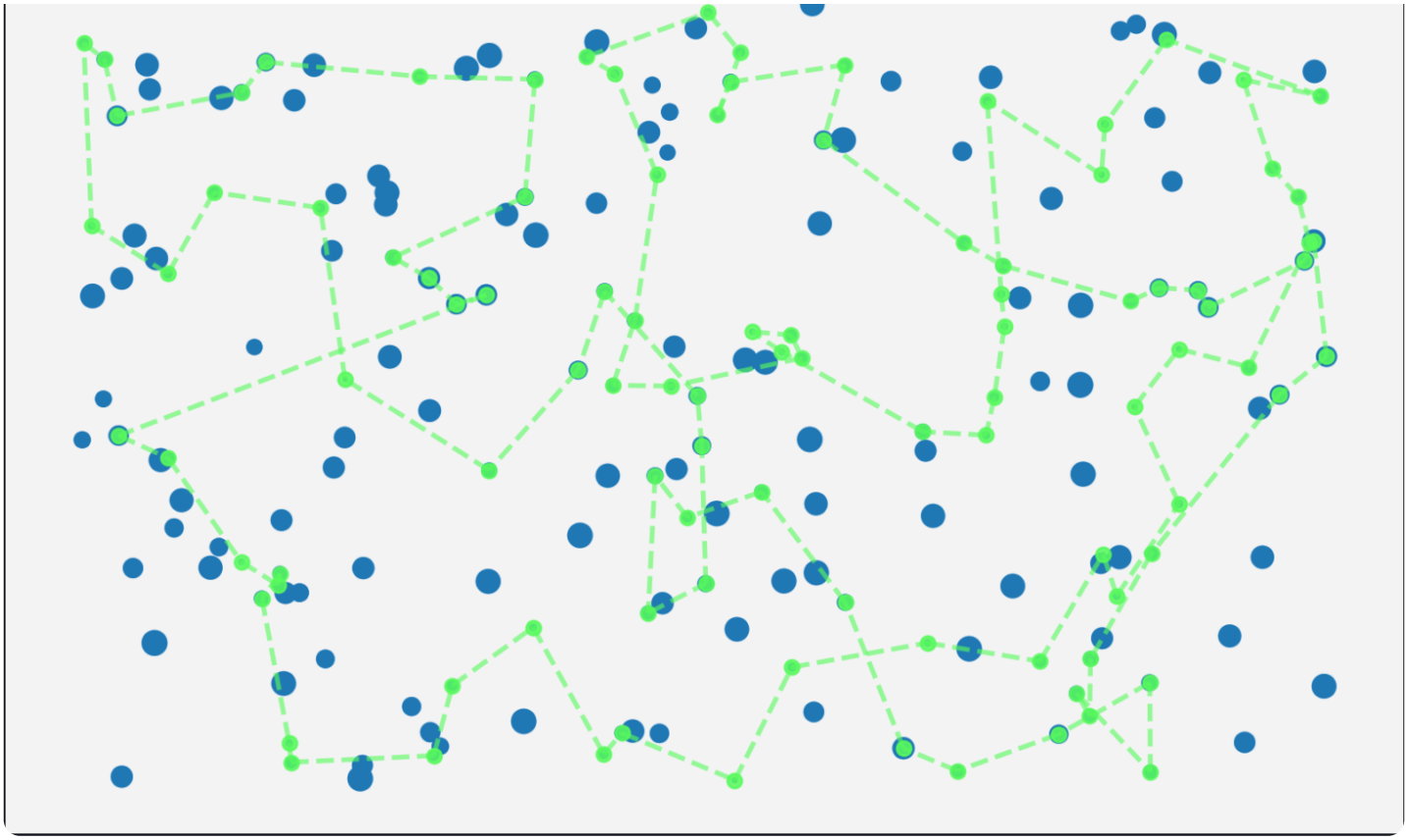
```
        append closest_point to tsp_path

        remove closest_point from not_visited_points

        last_point = closest_point

    return tsp_path
```

TSP A



Best found path: [124, 94, 63, 53, 180, 154, 135, 123, 65, 116, 59, 115, 139, 193, 41, 42, 160, 34, 22, 18, 108, 69, 159, 181, 184, 177, 54, 30, 48, 43, 151, 176, 80, 79, 133, 162, 51, 137, 183, 143, 0, 117, 46, 68, 93, 140, 36, 163, 199, 146, 195, 103, 5, 96, 118, 149, 131, 112, 4, 84, 35, 10, 190, 127, 70, 101, 97, 1, 152, 120, 78, 145, 185, 40, 165, 90, 81, 113, 175, 171, 16, 31, 44, 92, 57, 106, 49, 144, 62, 14, 178, 52, 55, 129, 2, 75, 86, 26, 100, 121]

TSP B



Best found path: [16, 1, 117, 31, 54, 193, 190, 80, 175, 5, 177, 36, 61, 141, 77, 153, 163, 176, 113, 166, 86, 185, 179, 94, 47, 148, 20, 60, 28, 140, 183, 152, 18, 62, 124, 106, 143, 0, 29, 109, 35, 33, 138, 11, 168, 169, 188, 70, 3, 145, 15, 155, 189, 34, 55, 95, 130, 99, 22, 66, 154, 57, 172, 194, 103, 127, 89, 137, 114, 165, 187, 146, 81, 111, 8, 104, 21, 82, 144, 160, 139, 182, 25, 121, 90, 122, 135, 63, 40, 107, 100, 133, 10, 147, 6, 134, 51, 98, 118, 74]

NN to any point

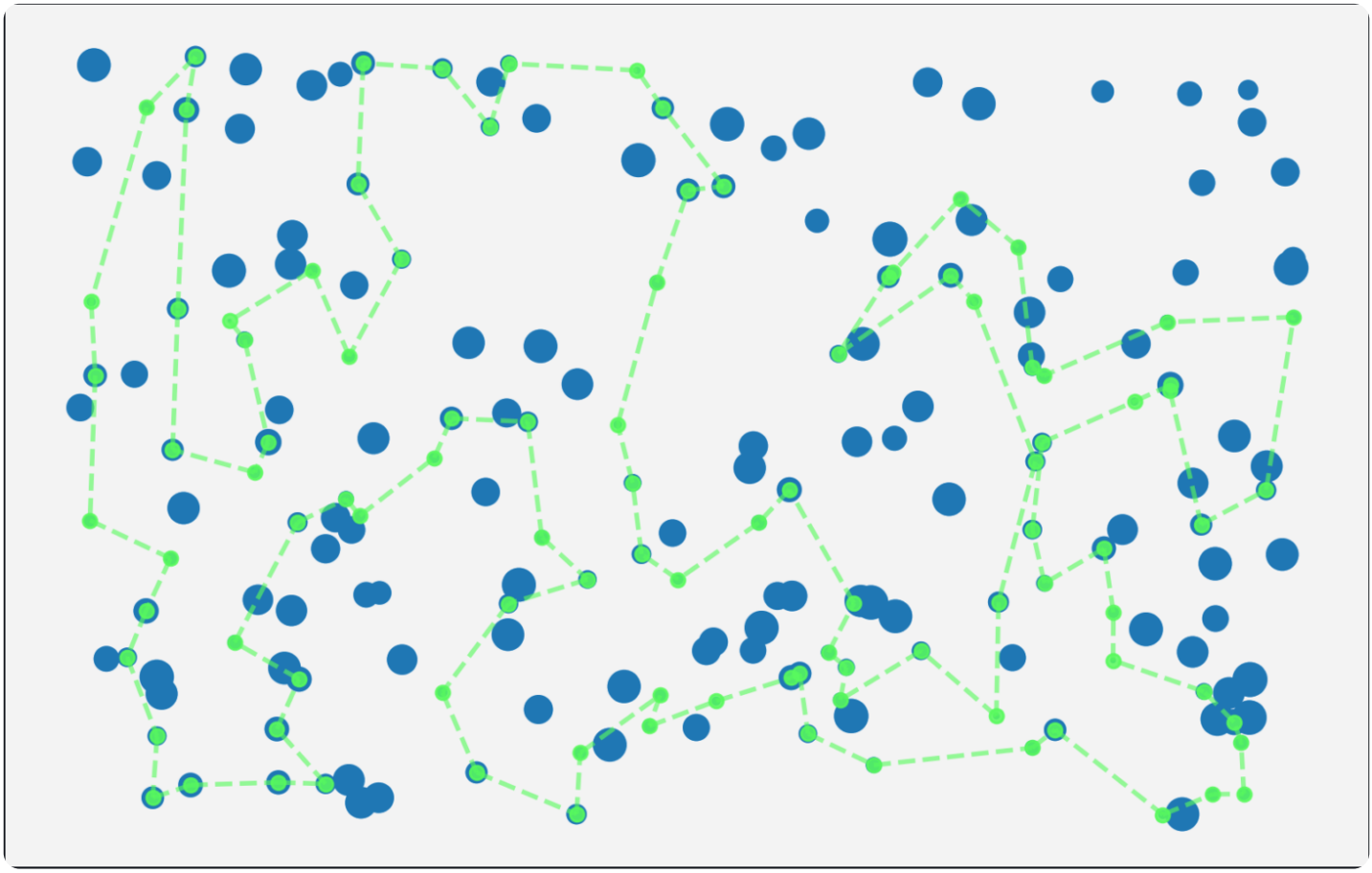
Pseudocode

```
find_closest(point, points, distance_matrix):
    closest_distance = INFINITY
    closest_point = point

    for each candidate_point in points:
        current_distance = distance_matrix[point, candidate_point] + candidate_po

        if current_distance < closest_distance:
            closest_distance = current_distance
            closest_point = candidate_point
```

```
    return closest_point, closest_distance



find_cheapest_extension(point_a, point_b, points, distance_matrix):
    closest_distance = infinity
    closest_point = point_a

    for each candidate_point in points:
        current_distance =
            distance_matrix[point_a, candidate_point] +
            distance_matrix[candidate_point, point_b] -
            distance_matrix[point_a, point_b] +
            candidate_point.cost

        if current_distance < closest_distance:
            closest_distance = current_distance
            closest_point = candidate_point

    return closest_point, closest_distance



greedy_nn_to_any_point(points, starting_point, distance_matrix):
    tsp_path = [starting_point]
    not_visited_points = points
    remove starting_point from not_visited_points

    for i from 1 to ceil(length(points) / 2):
        insert_spot = 0
        closest_point = 0
        closest_distance = infinity

        for pos from 0 to length(tsp_path):

            if pos == 0:
                point, distance = find_closest(tsp_path[0], not_visited_points, d

            else if pos == length(tsp_path):
                point, distance = find_closest(tsp_path[length(tsp_path) - 1], no

            else:
```

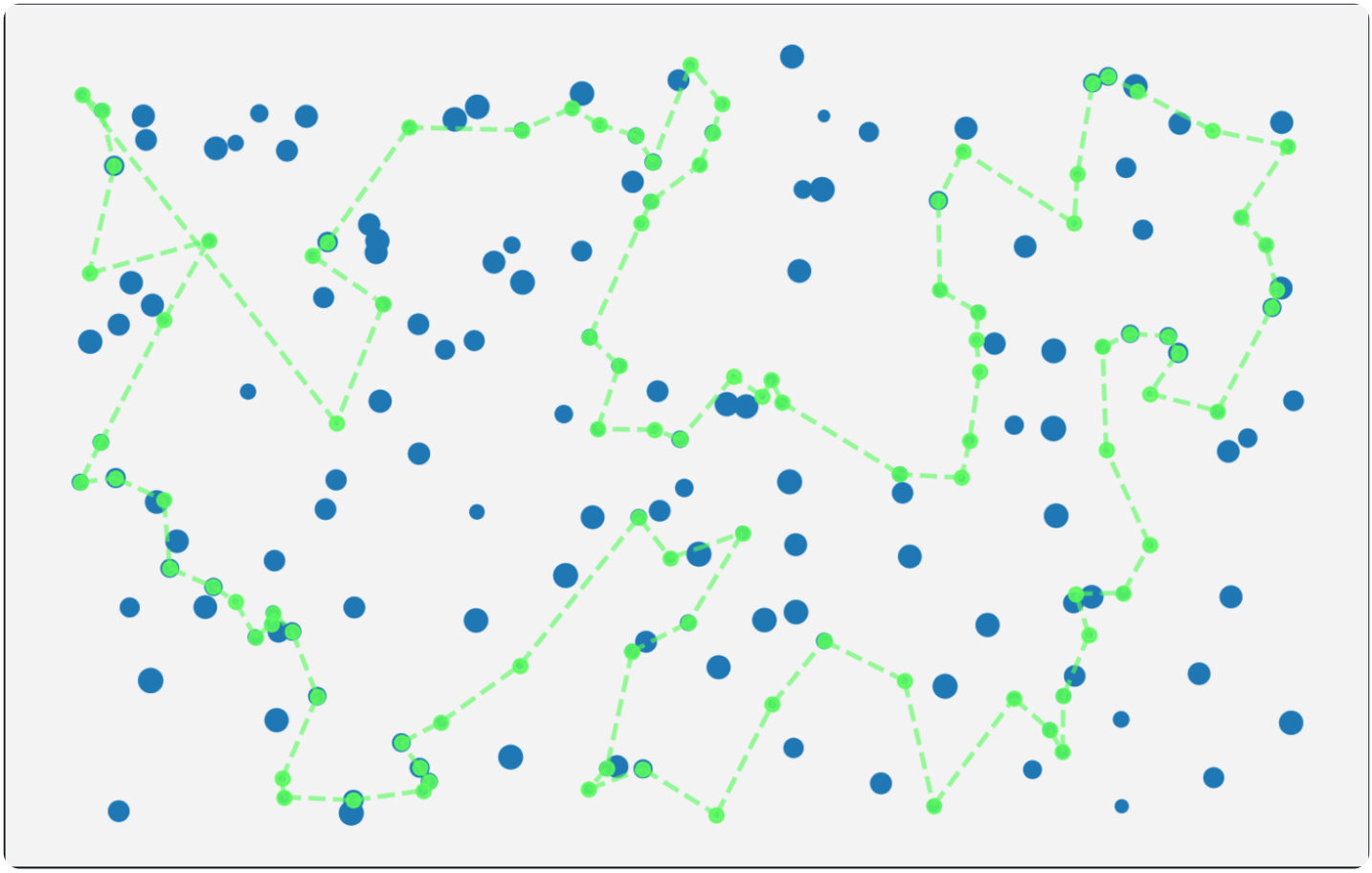
```
                a = tsp_path[pos - 1]
                b = tsp_path[pos]
                point, distance =
                    find_cheapest_extension(a, b, not_visited_points, distance_ma

            if distance < closest_distance:
                closest_distance = distance
                insert_spot = pos
                closest_point = point

        insert closest_point into tsp_path at insert_spot
        remove closest_point from not_visited_points

    return tsp_path
```

TSP A



Best found path: [68, 46, 115, 139, 193, 41, 5, 42, 181, 159, 69, 108, 18, 22, 146, 34, 160, 48, 54, 177, 10, 190, 4, 112, 84, 35, 184, 43, 116, 65, 59, 118, 51, 151, 133, 162, 123, 127, 70, 135, 180, 154, 53, 100, 26, 86, 75, 44, 25, 16, 171, 175, 113, 56, 31, 78, 145, 179, 92, 57, 52, 185, 119, 40, 196, 81, 90, 165, 106, 178, 14, 144, 62, 9, 148, 102, 49, 55, 129, 120, 2, 101, 1, 97, 152, 124, 94, 63, 79, 80, 176, 137, 23, 186, 89, 183, 143, 0, 117, 93]

TSP B



Best found path: [40, 107, 100, 63, 122, 135, 38, 27, 16, 1, 156, 198, 117, 193, 31, 54, 73, 136, 190, 80, 162, 175, 78, 142, 45, 5, 177, 104, 8, 111, 82, 21, 61, 36, 91, 141, 77, 81, 153, 187, 163, 89, 127, 103, 113, 176, 194, 166, 86, 95, 130, 99, 22, 185, 179, 66, 94, 47, 148, 60, 20, 28, 149, 4, 140, 183, 152, 170, 34, 55, 18, 62, 124, 106, 143, 35, 109, 0, 29, 160, 33, 138, 11, 139, 168, 195, 145, 15, 3, 70, 13, 132, 169, 188, 6, 147, 191, 90, 51, 121]

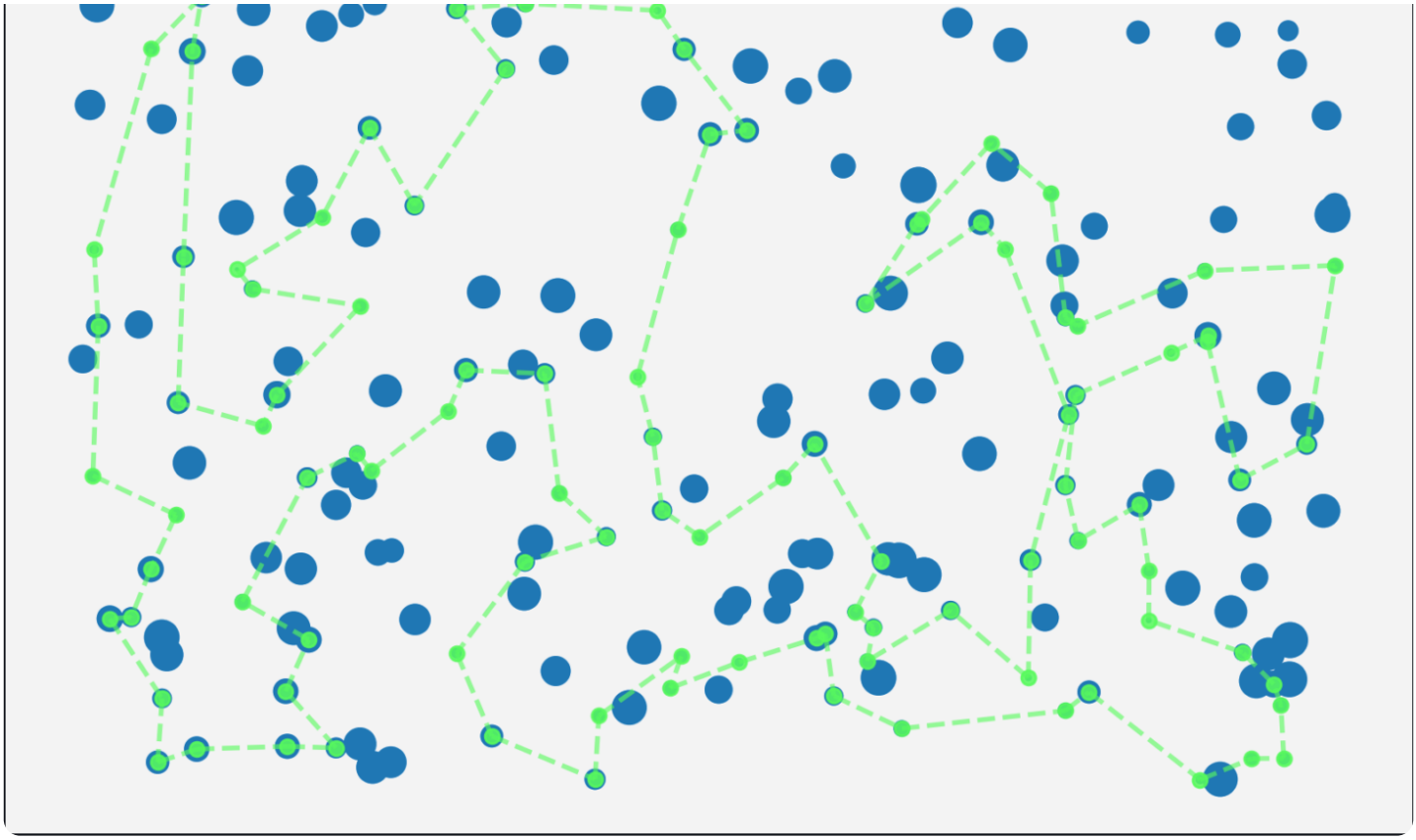
Greedy Cycle

Pseudocode

```
find_cheapest_extension(point_a, point_b, points, distance_matrix):
    closest_distance = infinity
    closest_point = point_a

    for each candidate_point in points:
        current_distance =
```

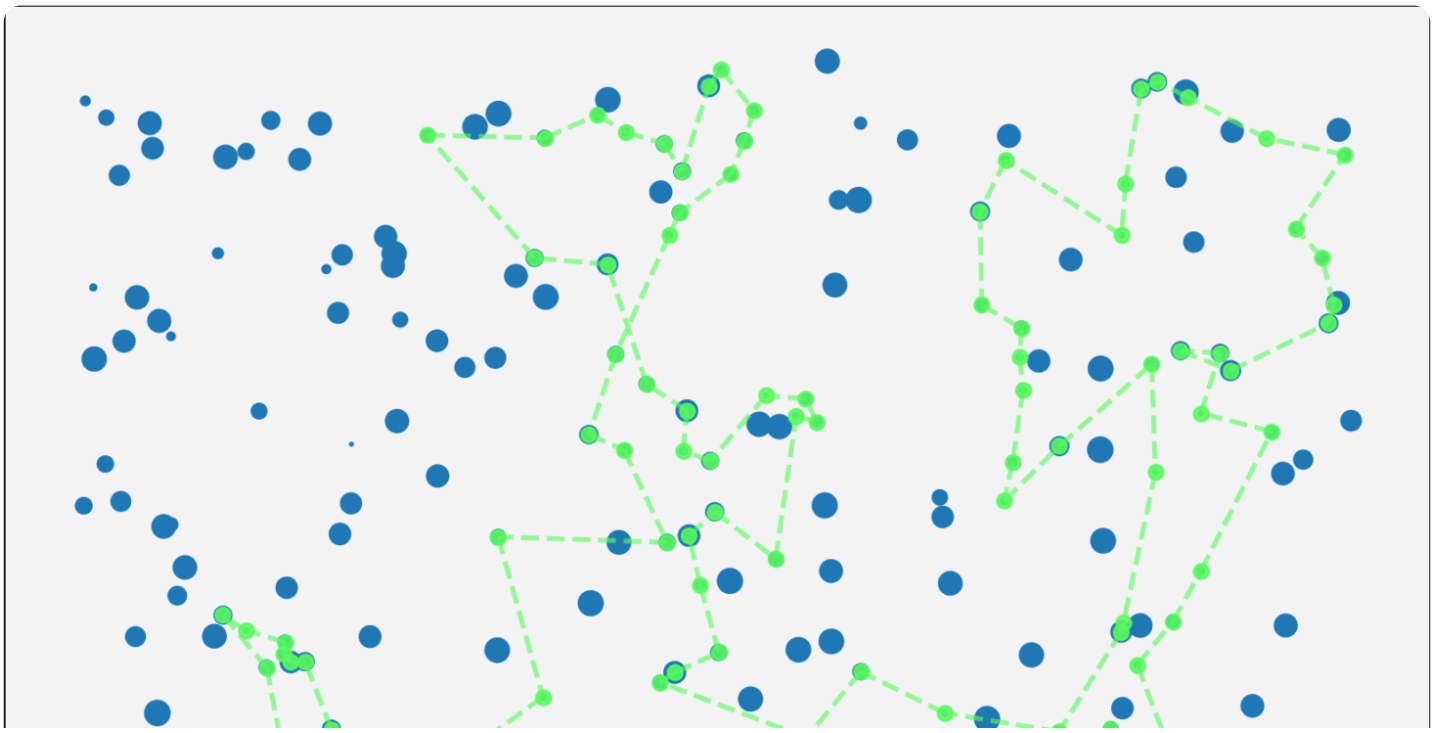
```
            distance_matrix[point_a, candidate_point] +
            distance_matrix[candidate_point, point_b] -
            distance_matrix[point_a, point_b] +
            candidate_point.cost

        if current_distance < closest_distance:
            closest_distance = current_distance
            closest_point = candidate_point

    return closest_point, closest_distance



greedy_cycle(points, starting_point, distance_matrix):
    tsp_path = [starting_point]
    not_visited_points = points
    remove starting_point from not_visited_points
    for i from 1 to ceil(length(points) / 2):
        insert_spot = 0
        closest_point = starting_point
        closest_distance = infinity
        for pos from 0 to length(tsp_path):
            if pos == 0 or pos == length(tsp_path):
                a = tsp_path[length(tsp_path) - 1]
                b = tsp_path[0]
            else:
                a = tsp_path[pos - 1]
                b = tsp_path[pos]

            point, distance = find_cheapest_extension(a, b, not_visited_points, d
            if distance < closest_distance:
                closest_distance = distance
                insert_spot = pos
                closest_point = point
        insert closest_point into tsp_path at insert_spot
        remove closest_point from not_visited_points
    return tsp_path
```

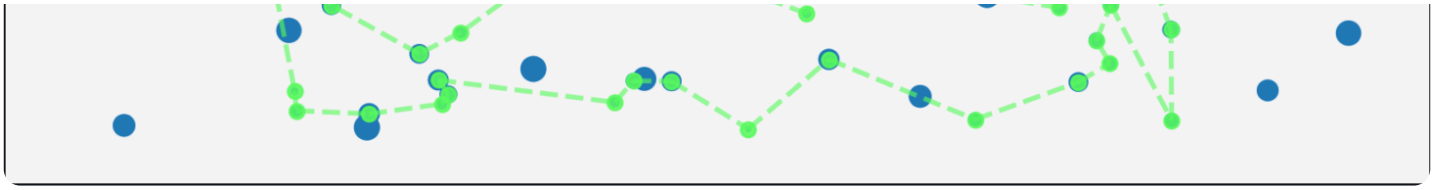
TSP A



Best found path: [46, 68, 139, 193, 41, 115, 5, 42, 181, 159, 69, 108, 18, 22, 146, 34, 160, 48, 54, 30, 177, 10, 190, 4, 112, 84, 35, 184, 43, 116, 65, 59, 118, 51, 151, 133, 162, 123, 127, 70, 135, 180, 154, 53, 100, 26, 86, 75, 44, 25, 16, 171, 175, 113, 56, 31, 78, 145, 179, 92, 57, 52, 185, 119, 40, 196, 81, 90, 165, 106, 178, 14, 144, 62, 9, 148, 102, 49, 55, 129, 120, 2, 101, 1, 97, 152, 124, 94, 63, 79, 80, 176, 137, 23, 186, 89, 183, 143, 117, 0]

TSP B





Best found path: [162, 175, 78, 142, 36, 61, 91, 141, 97, 187, 165, 127, 89, 103, 137, 114, 113, 194, 166, 179, 185, 99, 130, 22, 66, 94, 47, 148, 60, 20, 28, 149, 4, 140, 183, 152, 170, 34, 55, 18, 62, 124, 106, 128, 95, 86, 176, 180, 163, 153, 81, 77, 21, 87, 82, 8, 56, 144, 111, 0, 35, 109, 29, 160, 33, 49, 11, 43, 134, 147, 6, 188, 169, 132, 13, 161, 70, 3, 15, 145, 195, 168, 139, 182, 138, 104, 25, 177, 5, 45, 136, 73, 164, 31, 54, 117, 198, 193, 190, 80]

TSP A

	min	mean	max
Random	232312	265198.96	296002
NN to last point	83182	85108.51	89433
NN to any point	71179	73174.06	75450
Greedy Cycle	71488	72609.075	74410

TSP B

	min	mean	max
Random	191273	213945.54	236821
NN to last point	52319	54390.43	59030
NN to any point	44417	45870.255	53438
Greedy Cycle	48765	51301.73	57324

Conclusions

- **Random algorithm** = the fastest, but gives terrible results.

- **NN to last point** = looks like typical greedy algorithm, visible not optimal, multiple crosses, but much better than random with ridiculously short time.
- **NN to any point** = performs usually worse than Greedy Cycle, with few runs, which got better results (It's possible as they are greedy algorithms, so they can make mistakes and not backtrack them).
- **Greedy cycle** = very similar to **NN to any point**, much longer time than **NN to last point**, but visibly better result.